



CI1056 – ALGORITMOS E ESTRUTURA DE DADOS II

Profa. Rachel Reis
rachel@inf.ufpr.br



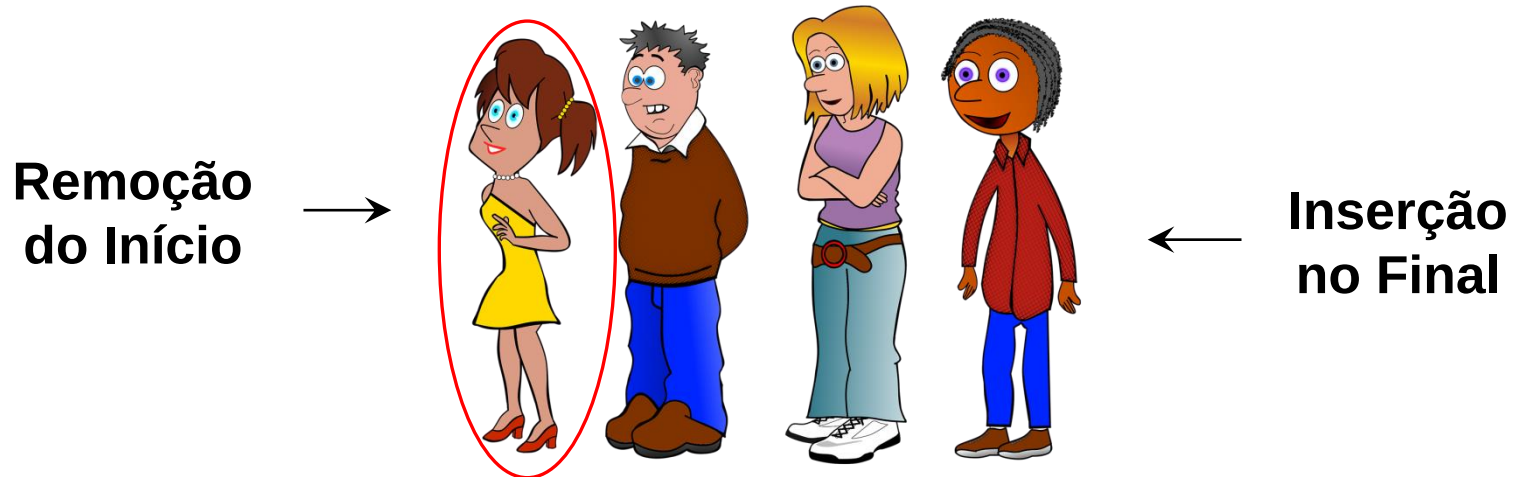
AULA DE HOJE

Filas de prioridade e Heaps

Adaptação do material gentilmente
disponibilizado pelos Profs. André
Vignatti e Marcos Castilho

RELEMBRANDO – TAD FILA

- Funcionamento (FIFO - *First In First Out*)

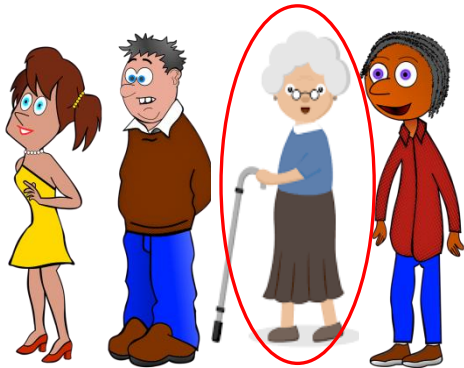


TAD FILA DE PRIORIDADE

- Operações de interesse:
 - **Inserir** um elemento.
 - **Remover o máximo** (elemento com maior prioridade).

TAD FILA DE PRIORIDADE

- Aplicações



Fila de banco



SO - Controle
de processos



Ex.: aplicativos
de celular

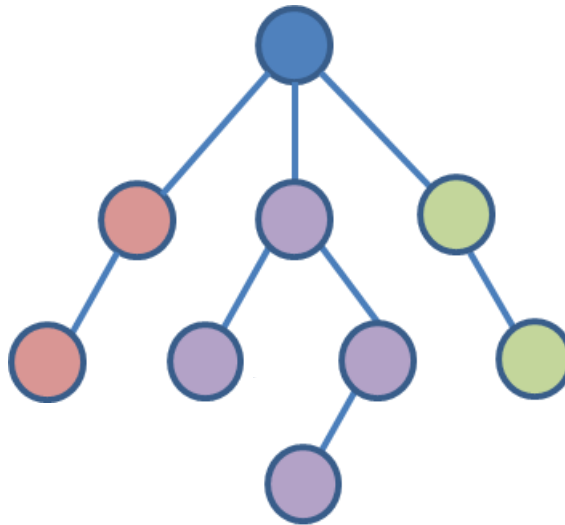
TAD FILA DE PRIORIDADE

- Existem diferentes formas de implementar as filas de prioridade. Utilizaremos a versão conhecida como *heap* binário.
- Um *heap* binário é uma árvore binária quase completa.

O que é uma Árvore?

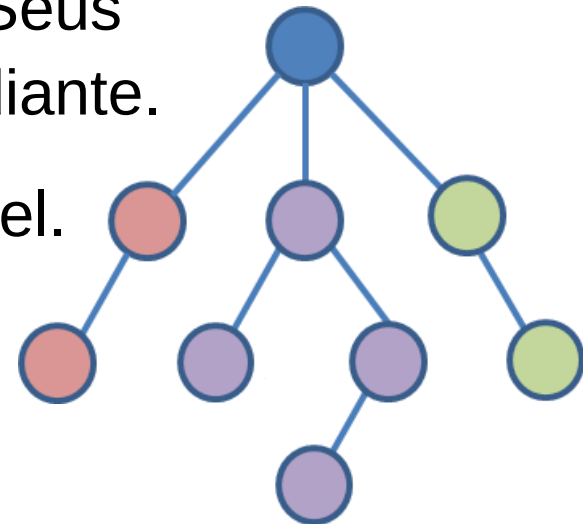
TAD ÁRVORE - DEFINIÇÃO

- É um tipo abstrato de dados caracterizado pela organização hierárquica entre seus elementos.
- É formado por um conjunto finito de nós (nodos, vértices) e arestas.



TAD ÁRVORE - TERMINOLOGIAS

- Raiz: primeiro nó de uma árvore que caracteriza o início da organização hierárquica.
- Folhas: nós que não possuem descendentes (filhos).
- Nível de um nó: o nó raiz tem nível 0. Seus descendentes tem nível 1 e assim por diante.
- Altura da árvore: igual ao seu maior nível.



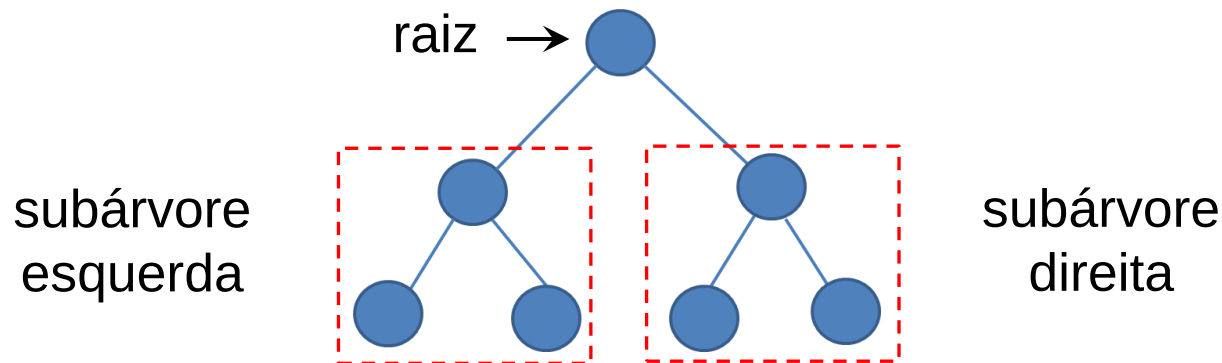
TAD FILA DE PRIORIDADE

- Existem diferentes formas de implementar as filas de prioridade. Utilizaremos a versão conhecida como *heap* binário.
- Um *heap* binário é uma árvore binária quase completa.

O que é uma Árvore Binária?

ÁRVORE BINÁRIA - DEFINIÇÃO

- Árvore em que todos os nós possuem 0 (zero), 1 (um) ou 2 (dois) descendentes.

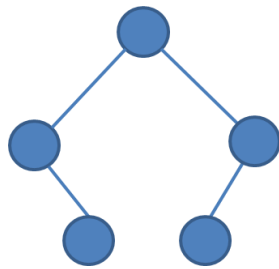


ÁRVORE BINÁRIA

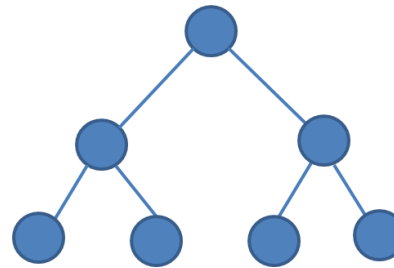
- Número **máximo de nós** n de uma árvore binária de altura $h \geq 0$

$$n = 2^{h+1} - 1 = 2^{2+1} - 1 = 7$$

- Exemplo:



5 elementos



Máximo: 7 elementos

TAD FILA DE PRIORIDADE

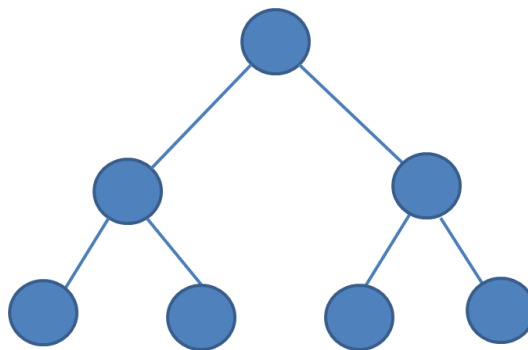
- Existem diferentes formas de implementar as filas de prioridade. Utilizaremos a versão conhecida como *heap* binário.
- Um *heap* binário é uma árvore binária (quase) completa.

**O que é uma Árvore Binária
(quase) completa?**

ÁRVORE BINÁRIA COMPLETA

1. Todos os nós possuem ou 0 (zero) ou 2 (dois) filhos.
2. Todos os nós folhas encontram-se no mesmo nível.

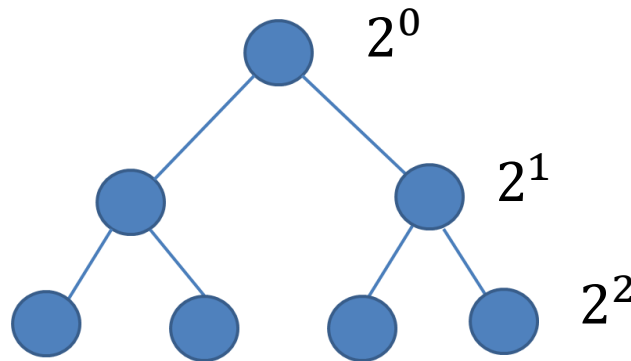
- Exemplo:



ÁRVORE BINÁRIA COMPLETA

Teorema 1. *Em uma árvore binária completa, $n = 2^{h+1} - 1$*

Demonstração. A quantidade de nós é igual a soma de nós em cada nível, ou seja, $2^0 + 2^1 + \dots + 2^h$. Usando a fórmula da p.g. finita, obtemos o resultado desejado.



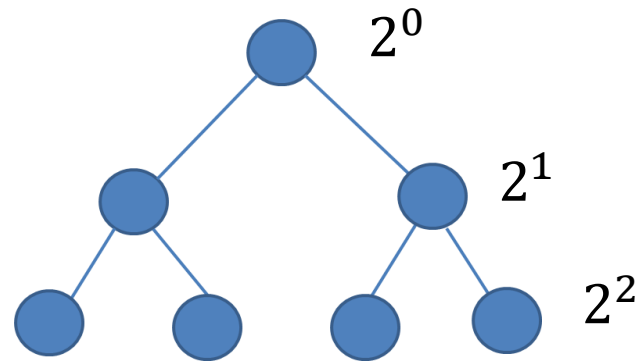
USANDO P.G. FINITA

$$S_k = a_1 * \frac{c^k - 1}{c - 1}$$

$$a_1 = 2^0 \text{ (1}^0 \text{ termo)}$$

$$c = 2 \text{ (razão)}$$

$$k = 3 \text{ (qtde de níveis completos)}$$

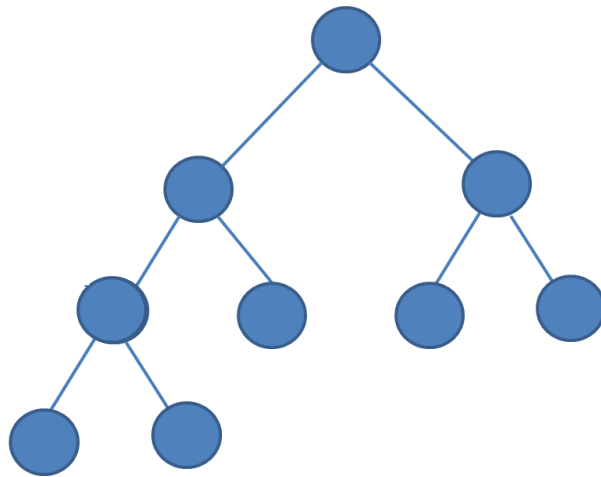


$$S_k = 1 * \frac{2^3 - 1}{2 - 1} = \frac{8 - 1}{1} = 7$$

ÁRVORE BINÁRIA QUASE COMPLETA

Uma árvore binária quase completa é uma árvore binária completa sem alguns dos nós do **último nível**.

- Exemplo:



ÁRVORE BINÁRIA QUASE COMPLETA

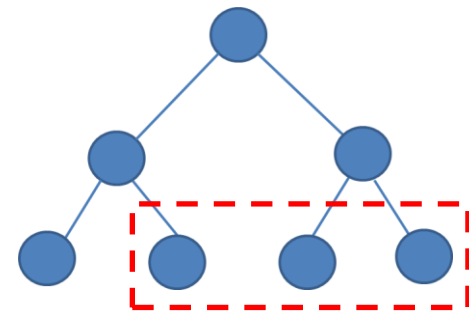
Teorema 2. *Em uma árvore binária quase completa,*

$$n \geq 2^h \quad \text{e} \quad n < 2^{h+1} - 1$$

Demonstração. Partindo de uma árvore binária completa, podemos retirar no máximo $2^h - 1$ nós sem alterar a altura da árvore, obtendo uma árvore binária quase completa.

RACIOCINANDO

$h = 2$



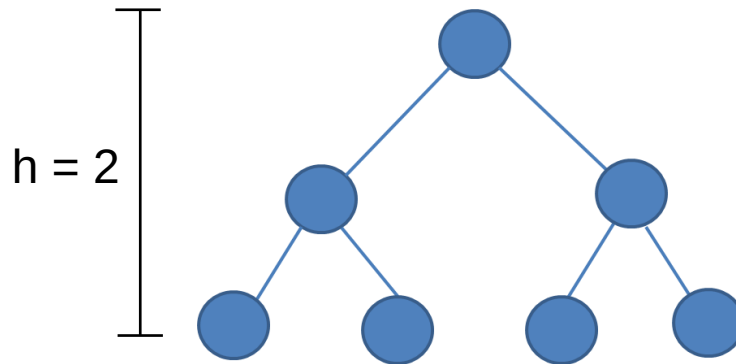
- $$\begin{aligned}n &= 2^{h+1} - 1 \\2^{h+1} - 1 - (2^h - 1) \\2^{h+1} - 1 - 2^h + 1 \\2^{h+1} - 2^h \\2^h \cdot 2^1 - 2^h \\2^h(2^1 - 1) \\n &= 2^h\end{aligned}$$

Número de nós de uma árvore binária quase completa é **pelo menos** 2^h .

ÁRVORE BINÁRIA QUASE COMPLETA

Teorema 3. *Em uma árvore binária (quase) completa $h = \lfloor \log_2 n \rfloor$*

Para provar esse teorema, temos que uma árvore binária completa de altura h tem exatamente $n = 2^{h+1} - 1$ nós.



RACIOCINANDO

- $n = 2^{h+1} - 1 \rightarrow 2^{h+1} = n + 1$
 $\log_2 2^{h+1} = \log_2(n + 1)$
 $\log_2 2^h \cdot 2^1 = \log_2(n + 1)$
 $\log_2 2^h + \log_2 2^1 = \log_2(n + 1)$
 $h \cdot \log_2 2 + 1 \cdot \log_2 2 = \log_2(n + 1)$
 $h \cdot 1 + 1 = \log_2(n + 1)$
 $h = \log_2(n + 1) - 1$

RACIOCINANDO

- $n = 2^{h+1} - 1 \rightarrow 2^{h+1} = n + 1$

$\vdots$

$$h = \log_2(n + 1) - 1$$

$\log_2(n + 1)$ é inteiro, pois, pelo Teorema 2, $n = 2^{h+1} - 1$.
Logo, podemos escrever $\log_2(n + 1) = \lfloor \log_2(n) \rfloor + 1$

$$h = \lfloor \log_2(n) \rfloor + 1 - 1$$

$$h = \lfloor \log_2(n) \rfloor$$

ÁRVORE BINÁRIA QUASE COMPLETA

Teorema 3. *Em uma árvore binária (quase) completa $h = \lfloor \log_2 n \rfloor$*

Para provar esse teorema, temos que uma árvore binária completa de altura h tem exatamente $n = 2^{h+1} - 1$ nós.

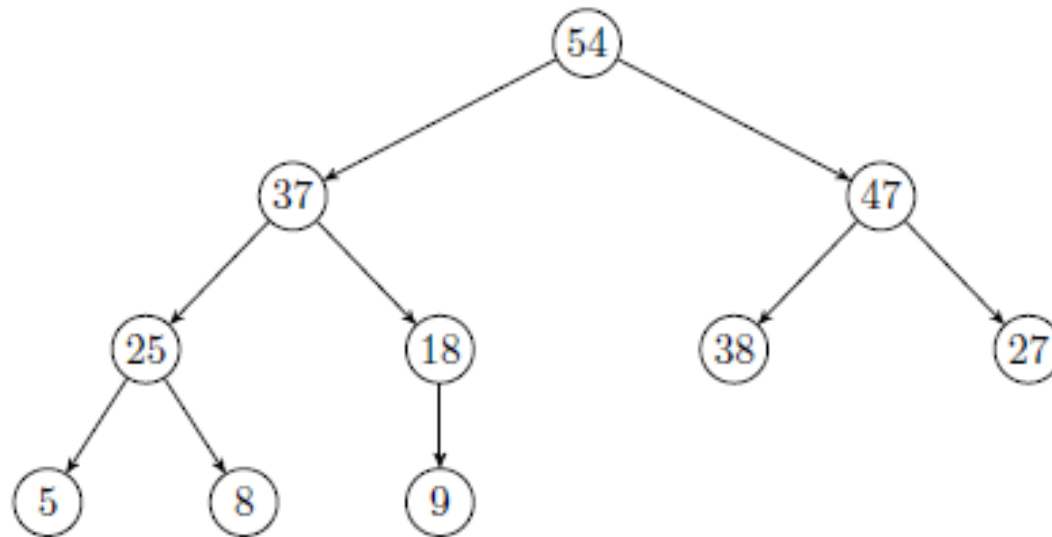
A altura de uma árvore binária quase completa é **igual** a altura de uma árvore binária completa.

HEAP BINÁRIO

- Um *heap* binário é uma árvore binária (quase) completa.
- **Tipos** de *heap*:
 - *Heap* de máximo
 - *Heap* de mínimo

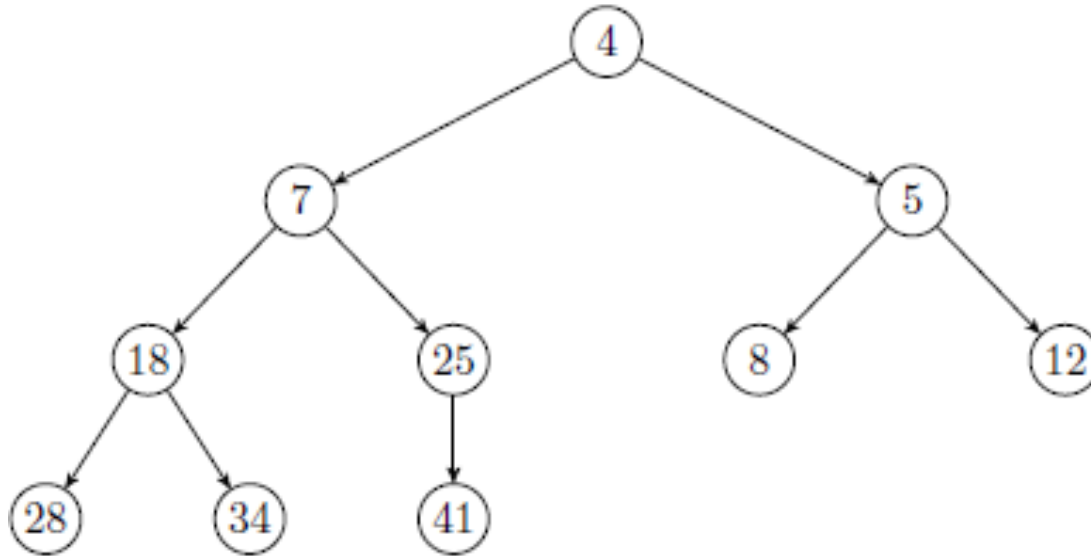
HEAP DE MÁXIMO

- Não existe ordem relativa entre os elementos do mesmo nível.
- O **elemento raiz** de uma subárvore é **maior** que todos os seus descendentes.



HEAP DE MÍNIMO

- Não existe ordem relativa entre os elementos do mesmo nível.
- O **elemento raiz** de uma subárvore é **menor** que todos os seus descendentes.

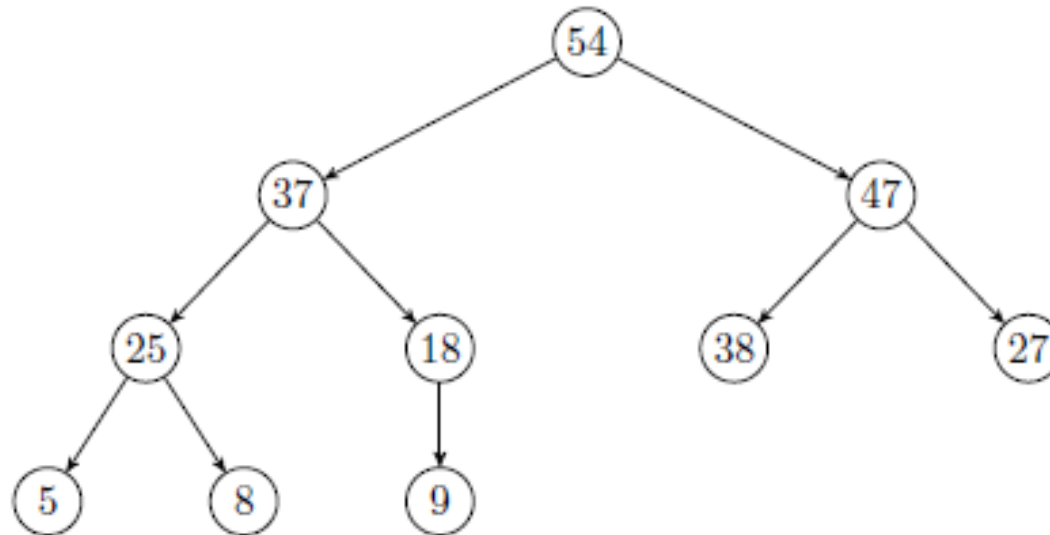


HEAP BINÁRIO - VETOR

- É possível representar um *heap* usando vetores.
- O primeiro elemento do vetor (índice 1) contém o nó raiz.
- Todos os outros elementos têm a seguinte propriedade:
 - $pai(i) = \left\lfloor \frac{i}{2} \right\rfloor$
 - $filho_{esq}(i) = 2i$
 - $filho_{dir}(i) = 2i + 1$
- Estas operações são extremamente rápidas nos computadores.

EXEMPLO – HEAP DE MÁXIMO

1	2	3	4	5	6	7	8	9	10
54	37	47	25	18	38	27	5	8	9

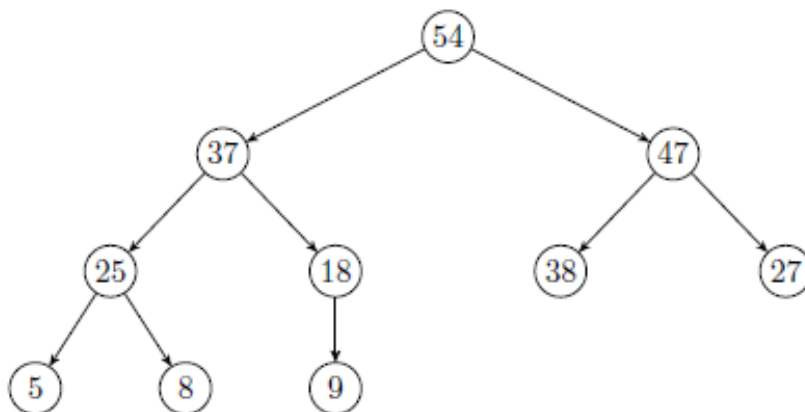


PROPRIEDADE - HEAP DE MÁXIMO

- Seja $pai(i)$ o índice do pai do nó i em um *heap* de máximo.

Dado um vetor A , então $A[pai(i)] \geq A[i]$, $\forall i > 1$.

- Assim, o **maior** elemento do vetor está no nó raiz ($A[1]$).

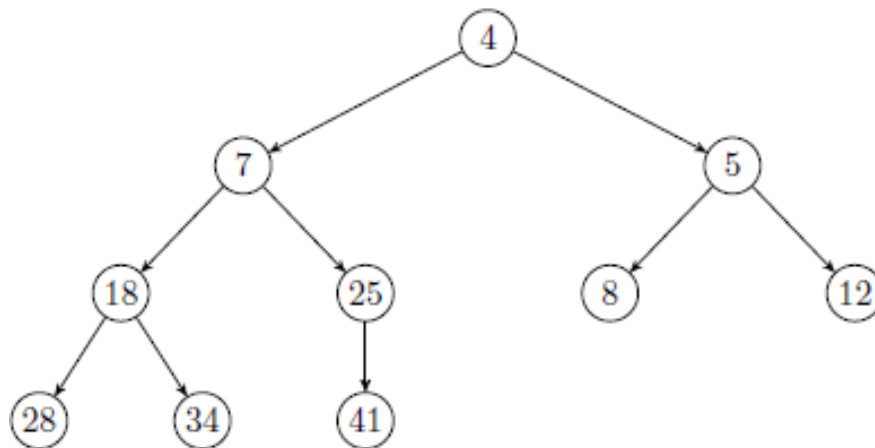


PROPRIEDADE - HEAP DE MÍNIMO

- Seja $pai(i)$ o índice do pai do nó i em um *heap* de mínimo.

Dado um vetor A , então $A[pai(i)] \leq A[i]$, $\forall i > 1$.

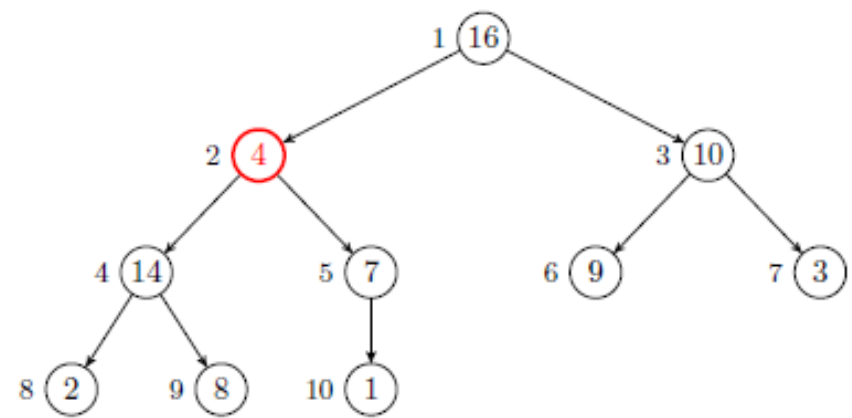
- Assim, o **menor** elemento do vetor está no nó raiz ($A[1]$).



ALGORITMO

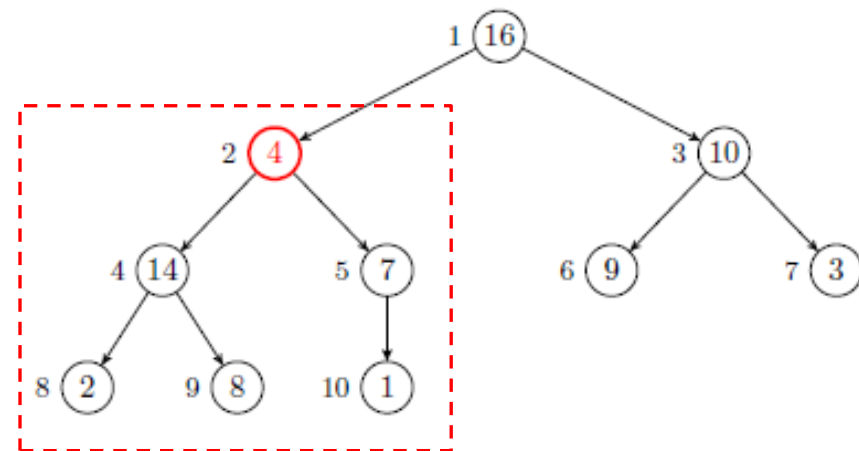
- Quando trabalhamos com *heap*, existem duas operações básicas:
 - **Inserir** um elemento no *heap*.
 - **Remover o máximo** do *heap*.
- Max-heapify: algoritmo usado para manter a propriedade de um *heap* (no caso aqui, *do heap* de máximo).

MAX-HEAPIFY



- **Problema:** uma raiz viola a propriedade do *heap* (detém um valor menor que seus filhos)
- **Solução:**
 - trocar a raiz em questão com o filho que tiver o maior valor(direito ou esquerdo)
 - o processo se repete recursivamente sobre a subárvore (*sub-heap*) em que a raiz foi trocada (se ainda violar a propriedade do heap).

MAX-HEAPIFY



- Problema computacional

Arrumar heap

Instância: (v, i) , onde v é um vetor e i um índice do vetor $\in [1, n]$.

- $esquerda(i)$: armazena o índice do nó raiz da subárvore esquerda de i .
- $direita(i)$: armazena o índice do nó raiz da subárvore direita de i .
- $esquerda(i)$ e $direita(i)$ são *heaps* de máximo.
- Eventualmente, $v[i]$ pode ser menor que seus filhos, e para este nó, é aceitável que ele viole a propriedade de *heap* de máximo.

Resposta: retorna um *heap* com a propriedade de máximo, conforme definido acima.

2. Algoritmo recursivo

→ max-heapify(v, i)

$esq \leftarrow esquerda(i)$

$dir \leftarrow direita(i)$

Se $esq \leq n$ e $v[esq] > v[i]$

$maior \leftarrow esq$

Senão

$maior \leftarrow i$

Se $dir \leq n$ e $v[dir] > v[maior]$

$maior \leftarrow dir$

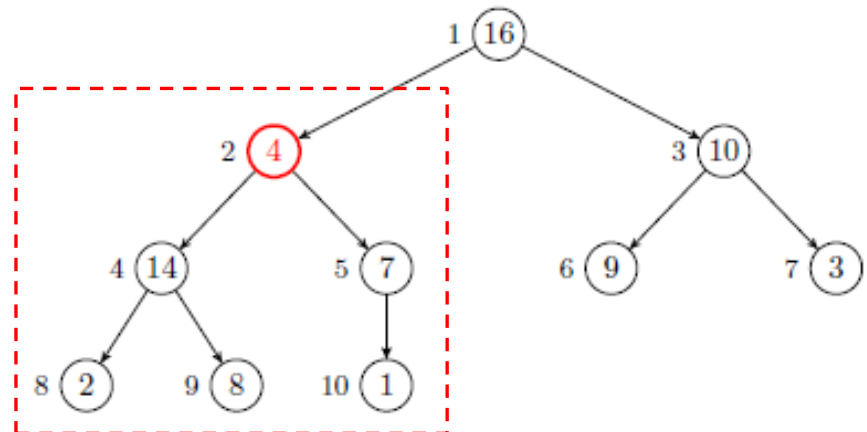
Se $maior \neq i$

Troca($v, i, maior$)

max-heapify($v, maior$)

	1	2	3	4	5	6	7	8	9	10
v	16	4	10	14	7	9	3	2	8	1

max-heapify ($v, 2$)



2. Algoritmo recursivo

max-heapify(v, i)

$esq \leftarrow esquerda(i)$

$dir \leftarrow direita(i)$

Se $esq \leq n$ e $v[esq] > v[i]$

$maior \leftarrow esq$

Senão

$maior \leftarrow i$

Se $dir \leq n$ e $v[dir] > v[maior]$

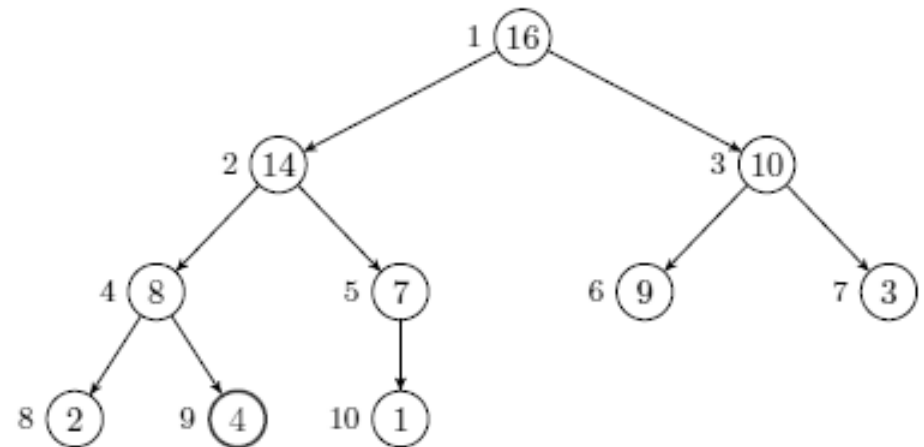
$maior \leftarrow dir$

Se $maior \neq i$

Troca($v, i, maior$)

max-heapify($v, maior$)

	1	2	3	4	5	6	7	8	9	10
v	16	14	10	8	7	9	3	2	4	1



ANÁLISE

- Análise do algoritmo recursivo.

$C(n)$: número de comparações entre os elementos do vetor de tamanho n .

ANÁLISE

Passo 1: Identificar as operações de interesse no algoritmo (que realizam comparações entre os elementos do vetor).

max-heapify(v, i)	
1	$esq \leftarrow esquerda(i)$
2	$dir \leftarrow direita(i)$
3	Se $esq \leq n$ e $v[esq] > v[i]$ ←
4	$maior \leftarrow esq$
5	Senão
6	$maior \leftarrow i$
7	Se $dir \leq n$ e $v[dir] > v[maior]$ ←
8	$maior \leftarrow dir$
9	Se $maior \neq i$
10	Troca($v, i, maior$)
11	max-heapify($v, maior$) ←

ANÁLISE

- Qual o melhor caso?

Duas comparações, pois o nó i já se encontra no lugar certo.

$$C^-(n) = 2$$

max-heapify(v, i)

$esq \leftarrow esquerda(i)$

$dir \leftarrow direita(i)$

Se $esq \leq n$ e $v[esq] > v[i]$ 

$maior \leftarrow esq$

Senão

$maior \leftarrow i$

Se $dir \leq n$ e $v[dir] > v[maior]$ 

$maior \leftarrow dir$

Se $maior \neq i$

Troca($v, i, maior$)

max-heapify($v, maior$)

ANÁLISE

- Qual o pior caso?

max-heapify(v, i)

$esq \leftarrow esquerda(i)$

$dir \leftarrow direita(i)$

Se $esq \leq n$ e $v[esq] > v[i]$

$maior \leftarrow esq$

Senão

$maior \leftarrow i$

Se $dir \leq n$ e $v[dir] > v[maior]$

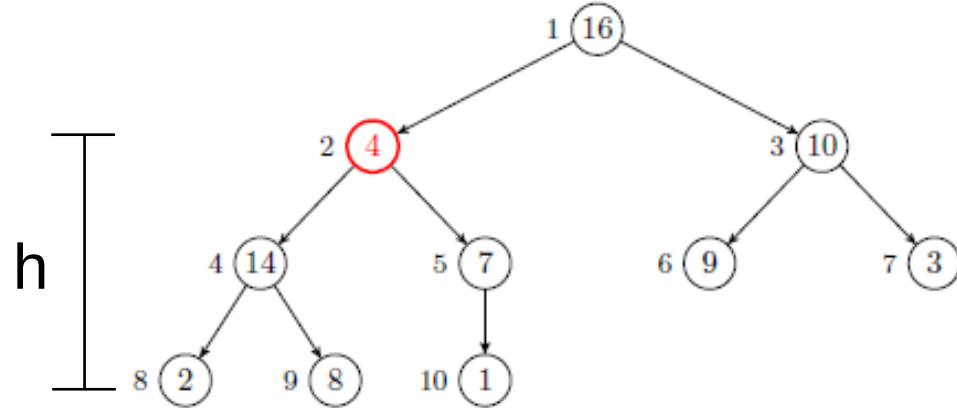
$maior \leftarrow dir$

Se $maior \neq i$

Troca($v, i, maior$)

max-heapify($v, maior$)

OBSERVAÇÃO



- O algoritmo max-heapify processa uma árvore iniciando em algum nó i que está em qualquer lugar da árvore.
- O pior caso é quando todas as recursões são feitas do nó i até chegar em uma folha. Logo, nossa análise depende da **altura da árvore**.

ANÁLISE

- Relação de recorrência.

Caso não base:

$$C^+(h) = \begin{cases} 0 & \text{se } h = 0 \\ 2 + C^+(h - 1) & \text{se } h > 0 \end{cases}$$

max-heapify(v, i)

$esq \leftarrow esquerda(i)$

$dir \leftarrow direita(i)$

Se $esq \leq n$ e $v[esq] > v[i]$ 

$maior \leftarrow esq$

Senão

$maior \leftarrow i$

Se $dir \leq n$ e $v[dir] > v[maior]$ 

$maior \leftarrow dir$

Se $maior \neq i$

Troca($v, i, maior$)

max-heapify($v, maior$) 

ANÁLISE

- Resolver a relação de recorrência.

$$C^+(h) = \begin{cases} 0 & \text{se } h = 0 \\ 2 + C^+(h - 1) & \text{se } h > 0 \end{cases}$$

- Solução:
 - $C^+(h) = 2h$ (fica como exercício)
 - $C^+(n) = 2 \cdot \lfloor \log_2 n \rfloor$